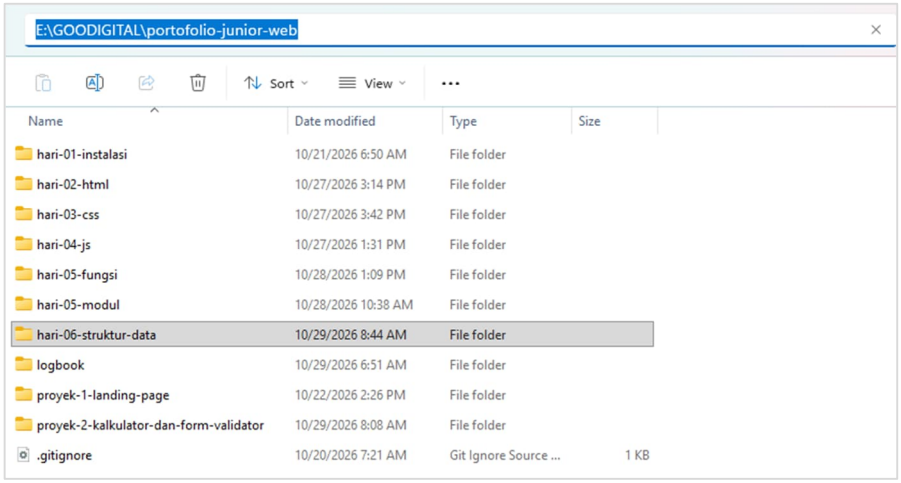
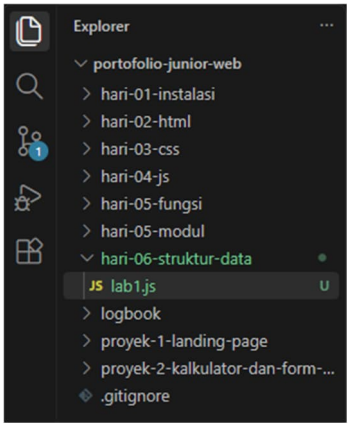
Modul Hari 6 - Struktur Data I: Array, Objek, Stack, dan Queue

Lab 1 - Array dan Objek (Data Peserta)

1. Data Peserta

- Membuat folder hari-06-struktur-data.
- Membuat file lab1.js.
- Membuat array data peserta.
- Menambahkan data peserta hingga minimal 10 peserta.
- Menyiapkan data untuk pengolahan array.







2. Peserta Aktif (filter)

- Membuat fungsi untuk menampilkan peserta aktif.
- Menggunakan method filter().
- Menguji hasil penyaringan data.

```
function tampilkanPesertaAktif(data) {
  return data.filter((p) => p.aktif);
}

console.log('Peserta Aktif');
console.log(
  tampilkanPesertaAktif(peserta)
);
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node lab1.js
Peserta Aktif
[
  { id: 1, nama: 'Levan', kelas: 'Web', nilai: 84, aktif: true },
  { id: 2, nama: 'Aina', kelas: 'Web', nilai: 86, aktif: true },
  { id: 4, nama: 'Amri', kelas: 'Web', nilai: 97, aktif: true },
  { id: 5, nama: 'Badru', kelas: 'Web', nilai: 100, aktif: true },
  { id: 6, nama: 'Abdul', kelas: 'Data', nilai: 95, aktif: true },
  { id: 7, nama: 'Tamam', kelas: 'Data', nilai: 87, aktif: true },
  { id: 8, nama: 'Azzam', kelas: 'Web', nilai: 75, aktif: true },
  { id: 10, nama: 'Haura', kelas: 'Web', nilai: 99, aktif: true }
]
```

3. Urutan Peserta Berdasarkan Nama

- Membuat fungsi pengurutan data peserta.
- Menggunakan sort() dan localeCompare().
- Menampilkan hasil pengurutan tanpa mengubah array asli.

```
function urutNama(data) {
  return [...data].sort(
    (a, b) => a.nama.localeCompare(b.nama)
  );
}

console.log('\nUrut Berdasarkan Nama');
console.log(
  urutNama(peserta)
);
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node lab1.js
]

Urut Berdasarkan Nama
[
  { id: 6, nama: 'Abdul', kelas: 'Data', nilai: 95, aktif: true },
  { id: 2, nama: 'Aina', kelas: 'Web', nilai: 86, aktif: true },
  { id: 4, nama: 'Amri', kelas: 'Web', nilai: 97, aktif: true },
  { id: 8, nama: 'Azzam', kelas: 'Web', nilai: 75, aktif: true },
  { id: 5, nama: 'Badru', kelas: 'Web', nilai: 100, aktif: true },
  { id: 3, nama: 'Dia', kelas: 'Data', nilai: 40, aktif: false },
  { id: 10, nama: 'Haura', kelas: 'Web', nilai: 99, aktif: true },
  { id: 1, nama: 'Levan', kelas: 'Web', nilai: 84, aktif: true },
  { id: 7, nama: 'Tamam', kelas: 'Data', nilai: 87, aktif: true },
  { id: 9, nama: 'Yaqin', kelas: 'Data', nilai: 56, aktif: false }
]
```

4. Cari Peserta Berdasarkan ID

- Membuat fungsi pencarian peserta berdasarkan ID.
- Menggunakan find().
- Menangani kondisi data tidak ditemukan.

```
function cariPeserta(id) {
  const hasil = peserta.find(
    (p) => p.id === id
  );

  if (!hasil) {
    return 'Peserta tidak ditemukan';
  }

  return hasil;
}

console.log('\nCari ID 5');
console.log(cariPeserta(5));

console.log('\nCari ID 99');
console.log(cariPeserta(99));
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node lab1.js
{ id: 5, nama: 'Badru', kelas: 'Web', nilai: 100, aktif: true }

Cari ID 99
Peserta tidak ditemukan
```

5. Jumlah Peserta per Kelas

- Menghitung jumlah peserta pada setiap kelas.
- Menggunakan objek sebagai penghitung.
- Menampilkan hasil perhitungan.

```
function hitungPerKelas(data) {

  const perKelas = {};

  for (const peserta of data) {
    perKelas[peserta.kelas] =
      (perKelas[peserta.kelas] || 0) + 1;
  }

  return perKelas;
}

console.log('\nJumlah Peserta per Kelas');
console.log(
  hitungPerKelas(peserta)
);
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node lab1.js
Jumlah Peserta per Kelas
{ Web: 6, Data: 4 }
```

6. Tambah, Ubah, dan Hapus Peserta

- Menambahkan data peserta baru.
- Mengubah data peserta.
- Menghapus data peserta.
- Menampilkan hasil perubahan data.

```
const dataPeserta = [...peserta];

dataPeserta.push({
  id: 11,
  nama: 'Quo',
  kelas: 'Data',
  nilai: 64,
  aktif: true
});

console.log('\nSetelah Tambah Peserta');
console.log(dataPeserta);

dataPeserta[2].nilai = 65;

console.log('\nSetelah Ubah Nilai Dia');
console.log(dataPeserta[2]);

const setelahHapus =
  dataPeserta.filter(
    (p) => p.id !== 9
  );

console.log('\nSetelah Hapus ID 9');
console.log(setelahHapus);
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node lab1.js

Setelah Tambah Peserta
[
  { id: 1, nama: 'Levan', kelas: 'Web', nilai: 84, aktif: true },
  { id: 2, nama: 'Aina', kelas: 'Web', nilai: 86, aktif: true },
  { id: 3, nama: 'Dia', kelas: 'Data', nilai: 40, aktif: false },
  { id: 4, nama: 'Amri', kelas: 'Web', nilai: 97, aktif: true },
  { id: 5, nama: 'Badru', kelas: 'Web', nilai: 100, aktif: true },
  { id: 6, nama: 'Abdul', kelas: 'Data', nilai: 95, aktif: true },
  { id: 7, nama: 'Tamam', kelas: 'Data', nilai: 87, aktif: true },
  { id: 8, nama: 'Azzam', kelas: 'Web', nilai: 75, aktif: true },
  { id: 9, nama: 'Yaqin', kelas: 'Data', nilai: 56, aktif: false },
  { id: 10, nama: 'Haura', kelas: 'Web', nilai: 99, aktif: true },
  { id: 11, nama: 'Quo', kelas: 'Data', nilai: 64, aktif: true }
]

Setelah Ubah Nilai Dia
{ id: 3, nama: 'Dia', kelas: 'Data', nilai: 65, aktif: false }

Setelah Hapus ID 9
[
  { id: 1, nama: 'Levan', kelas: 'Web', nilai: 84, aktif: true },
  { id: 2, nama: 'Aina', kelas: 'Web', nilai: 86, aktif: true },
  { id: 3, nama: 'Dia', kelas: 'Data', nilai: 65, aktif: false },
  { id: 4, nama: 'Amri', kelas: 'Web', nilai: 97, aktif: true },
  { id: 5, nama: 'Badru', kelas: 'Web', nilai: 100, aktif: true },
  { id: 6, nama: 'Abdul', kelas: 'Data', nilai: 95, aktif: true },
  { id: 7, nama: 'Tamam', kelas: 'Data', nilai: 87, aktif: true },
  { id: 8, nama: 'Azzam', kelas: 'Web', nilai: 75, aktif: true },
  { id: 10, nama: 'Haura', kelas: 'Web', nilai: 99, aktif: true },
  { id: 11, nama: 'Quo', kelas: 'Data', nilai: 64, aktif: true }
]
```

7. Ubah Array Menjadi Objek Berindeks ID

- Membuat objek berindeks ID menggunakan reduce().
- Melakukan pencarian data berdasarkan ID.
- Menguji akses data dengan indeks ID.

```
const pesertaById = peserta.reduce(
  (objek, p) => {

    objek[p.id] = p;

    return objek;

  },
  {}
);

console.log('\nObjek Berindeks ID');
console.log(pesertaById);

console.log('\nPeserta ID 5');
console.log(pesertaById[5]);

console.log('\nPeserta ID 10');
console.log(pesertaById[10]);
```

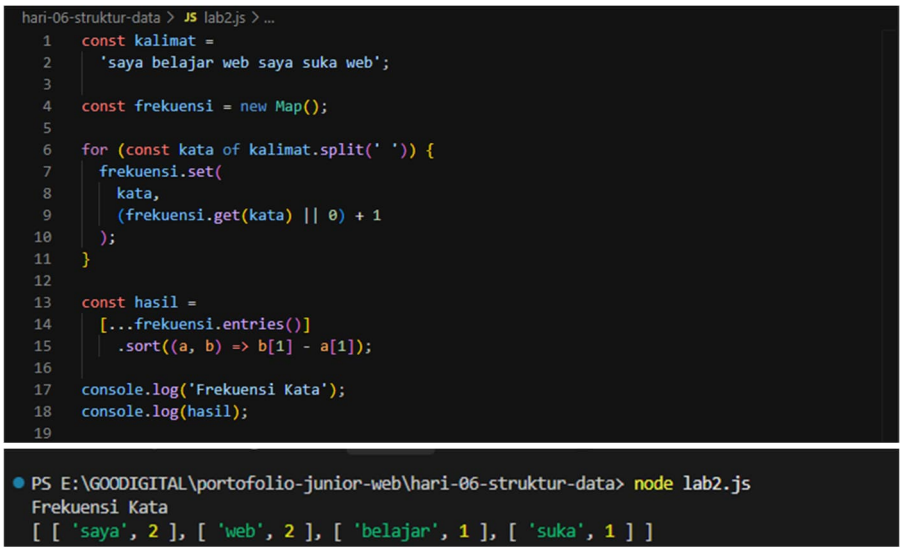
```
PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node lab1.js
Peserta ID 5
{ id: 5, nama: 'Badru', kelas: 'Web', nilai: 100, aktif: true }

Peserta ID 10
{ id: 10, nama: 'Haura', kelas: 'Web', nilai: 99, aktif: true }
```

Lab 2 - Map dan Set

1. Frekuensi Kata dengan Map

- Membuat file lab2.js.
- Menghitung frekuensi kemunculan kata menggunakan Map.
- Mengurutkan hasil berdasarkan frekuensi.
- Menampilkan hasil perhitungan.



```
hari-06-struktur-data > JS lab2.js > ...
1   const kalimat =
2     'saya belajar web saya suka web';
3
4   const frekuensi = new Map();
5
6   for (const kata of kalimat.split(' ')) {
7     frekuensi.set(
8       kata,
9       (frekuensi.get(kata) || 0) + 1
10    );
11  }
12
13  const hasil =
14    [...frekuensi.entries()]
15      .sort((a, b) => b[1] - a[1]);
16
17  console.log('Frekuensi Kata');
18  console.log(hasil);
19
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node lab2.js
Frekuensi Kata
[ [ 'saya', 2 ], [ 'web', 2 ], [ 'belajar', 1 ], [ 'suka', 1 ] ]
```

2. Hilangkan Duplikat Email dengan Set

- Membuat daftar email yang mengandung duplikat.
- Menggunakan Set untuk menghilangkan data duplikat.
- Menampilkan data sebelum dan sesudah diproses.



```
const email = [
  'amri@gmail.com',
  'badru@gmail.com',
  'amri@gmail.com',
  'haura@gmail.com',
  'badru@gmail.com'
];

console.log('\nSebelum Set');
console.log(email);

const emailUnik = [...new Set(email)];

console.log('\nSesudah Set');
console.log(emailUnik);
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node lab2.js
Sebelum Set
[
  'amri@gmail.com',
  'badru@gmail.com',
  'amri@gmail.com',
  'haura@gmail.com',
  'badru@gmail.com'
]

Sesudah Set
[ 'amri@gmail.com', 'badru@gmail.com', 'haura@gmail.com' ]
```

3. Irisan Dua Daftar Peserta dengan Set

- Membuat dua daftar peserta.
- Mencari anggota yang terdapat pada kedua daftar.
- Menampilkan hasil irisan data.

```
const kelasA = [
  'Amri',
  'Badru',
  'Aina',
  'Haura',
  'Tamam'
];

const kelasB = [
  'Badru',
  'Tamam',
  'Azzam',
  'Abdul',
  'Haura'
];

const setB = new Set(kelasB);

const irisan =
  kelasA.filter((nama) =>
    setB.has(nama)
  );

console.log('\nIrisan Kelas A dan B');
console.log(irisan);
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node lab2.js
[ 'Badru', 'Haura', 'Tamam' ]
```

4. Fungsi adaDuplikat

- Membuat fungsi pendeteksi data duplikat menggunakan Set.
- Menguji fungsi dengan array kosong, satu elemen, dan data berulang.

```
function adaDuplikat(daftar) {
  return new Set(daftar).size !== daftar.length;
}

console.log('\nArray Kosong');
console.log(adaDuplikat([]));

console.log('\nSatu Elemen');
console.log(adaDuplikat(['Amri']));

console.log('\nElemen Berulang');
console.log(
  adaDuplikat([
    'Amri',
    'Badru',
    'Amri'
  ])
);
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node lab2.js
Array Kosong
false

Satu Elemen
false

Elemen Berulang
true
```

Lab 3 - Stack

1. Struktur Data Stack

- Membuat file structures.js.
- Menambahkan class Stack.
- Mengekspor class menggunakan export.

hari-06-struktur-data > JS structures.js > ...

```
export class Stack {

  constructor() {
    this.items = [];
  }

  push(data) {
    this.items.push(data);
  }

  pop() {
    return this.isEmpty()
    ? undefined
    : this.items.pop();
  }

  peek() {
    return this.isEmpty()
    ? undefined
    : this.items[this.items.length - 1];
  }

  isEmpty() {
    return this.items.length === 0;
  }

}

```

2. Operasi Stack

- Membuat file lab3.js.
- Menambahkan data ke stack.
- Melihat elemen teratas stack.
- Menghapus elemen dari stack.
- Menguji stack kosong.

hari-06-struktur-data > JS lab3.js > ...

```
import { Stack } from './structures.js';

const tumpukan = new Stack();

tumpukan.push('A');
tumpukan.push('B');
tumpukan.push('C');

console.log('Peek');
console.log(tumpukan.peek());

console.log('\nPop 1');
console.log(tumpukan.pop());

console.log('\nPop 2');
console.log(tumpukan.pop());

console.log('\nPop 3');
console.log(tumpukan.pop());

console.log('\nPop Kosong');
console.log(tumpukan.pop());

```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node lab3.js
Peek
C

Pop 1
C

Pop 2
B

Pop 3
A

Pop Kosong
undefined
```

3. Validasi Tanda Kurung Seimbang

- Membuat fungsi validasi tanda kurung.
- Menggunakan struktur data Stack.
- Menguji beberapa variasi tanda kurung.

```
function kurungSeimbang(teks) {

  const pasangan = {
    ')': '(',
    ']': '[',
    '}': '{'
  };

  const tumpukan = new Stack();

  for (const karakter of teks) {

    if ('([{'.includes(karakter)) {
      tumpukan.push(karakter);
    }

    else if (karakter in pasangan) {

      if (
        tumpukan.isEmpty() ||
        tumpukan.pop() !== pasangan[karakter]
      ) {
        return false;
      }

    }

  }

  return tumpukan.isEmpty();
}
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node lab3.js
Kurung Seimbang
true
false
false
true
```

4. Balik String dan Periksa Palindrom

- Membalik string menggunakan Stack.
- Membuat fungsi pemeriksa palindrom.
- Menguji hasil fungsi.

```
function balikString(teks) {

  const tumpukan = new Stack();

  for (const karakter of teks) {
    tumpukan.push(karakter);
  }

  let hasil = '';

  while (!tumpukan.isEmpty()) {
    hasil += tumpukan.pop();
  }

  return hasil;
}

console.log('\nBalik String');

console.log(
  balikString('WEB')
);
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node lab3.js
Balik String
BEW

Palindrom
katak: true
web: false
malam: true
```

Lab 4 - Queue dan Simulasi Antrean Loket

1. Class Queue

- Menambahkan class Queue ke structures.js.
- Mengimplementasikan operasi antrean FIFO.
- Menggunakan export class Queue.

```
export class Queue {

  constructor() {
    this.items = {};
    this.head = 0;
    this.tail = 0;
  }

  enqueue(item) {
    this.items[this.tail] = item;
    this.tail++;
  }

  dequeue() {

    if (this.isEmpty()) {
      return undefined;
    }
```

2. Operasi Queue

- Membuat file queue-test.js.
- Menguji operasi enqueue(), dequeue(), dan front().
- Menguji queue kosong.

```
hari-06-struktur-data > JS queue-test.js > ...
1   import { Queue } from './structures.js';
2
3   const antrean = new Queue();
4
5   antrean.enqueue('A');
6   antrean.enqueue('B');
7   antrean.enqueue('C');
8
9   console.log('Front');
10  console.log(antrean.front());
11
12  console.log('\nDequeue 1');
13  console.log(antrean.dequeue());
14
15  console.log('\nDequeue 2');
16  console.log(antrean.dequeue());
17
18  console.log('\nDequeue 3');
19  console.log(antrean.dequeue());
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node queue-test.js
Front
A

Dequeue 1
A

Dequeue 2
B

Dequeue 3
C

Dequeue Kosong
undefined
```

3. Simulasi Antrean Loket

- Membuat file antrian.html.
- Membuat file antrian.js.
- Membuat antarmuka antrean loket.
- Menampilkan nomor yang sedang dipanggil dan jumlah antrean.

hari-06-struktur-data > antrian.html > ... Text Editor

```html
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Antrean Loket</title>
</head>
<body>

  <h1>Antrean Loket</h1>

  <p>
    Sedang dipanggil:
    <strong id="dipanggil">-</strong>
  </p>

  <p>
    Menunggu:
    <span id="jumlah">0</span>
  </p>

```

hari-06-struktur-data > JS antrian.js > ...

```js
import { Queue } from './structures.js';

const antrian = new Queue();

let nomorTerakhir = 0;

const dipanggil =
  document.getElementById('dipanggil');

const jumlah =
  document.getElementById('jumlah');

const daftar =
  document.getElementById('daftar');

function formatNomor(angka) {
  return 'A' + String(angka).padStart(3, '0');
}

function tampilkanDaftar() {

  daftar.textContent = '';

  for (
    let i = antrian.head;
    i < antrian.tail;
    i++
  ) {
```



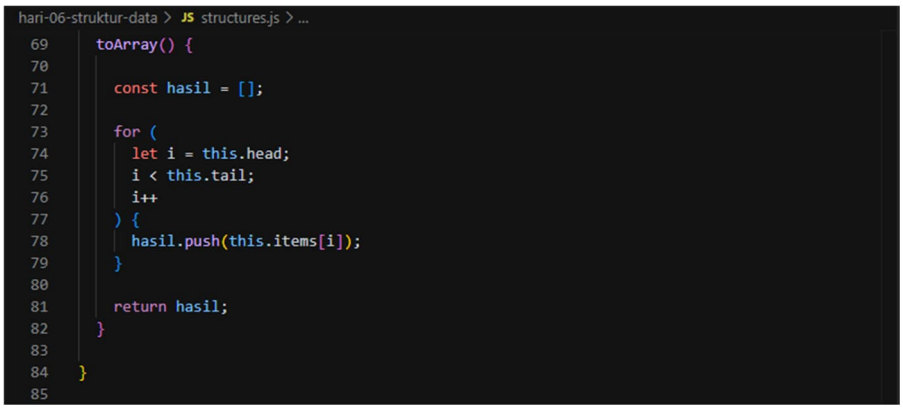






4. Catatan Desain Queue

- Meninjau implementasi Queue yang digunakan pada simulasi antrean.
- Menganalisis akses data Queue dari luar class.
- Mengidentifikasi peluang perbaikan desain.



hari-06-struktur-data > JS antrian.js > ...

```
20  function tampilkanDaftar() {
21
22    daftar.textContent = '';
23
24    for (const nomor of antrian.toArray()) {
25
26      const li =
27        document.createElement('li');
28
29      li.textContent = nomor;
30
31      daftar.append(li);
32    }
33
34    jumlah.textContent =
35      antrian.size();
36  }
```

Antrean Loket

Sedang dipanggil: -

Menunggu: 0

Ambil Nomor | Panggil Berikutnya

Antrean Loket

Sedang dipanggil: -

Menunggu: 3

Ambil Nomor | Panggil Berikutnya

1. A001
2. A002
3. A003

Antrean Loket

Sedang dipanggil: **A001**

Menunggu: 2

Ambil Nomor | Panggil Berikutnya

1. A002
2. A003

Antrean Loket

Sedang dipanggil: **Antrean kosong**

Menunggu: 0

Ambil Nomor | Panggil Berikutnya

Lab 5 - Mengukur Efisiensi

1. Program Pengukuran

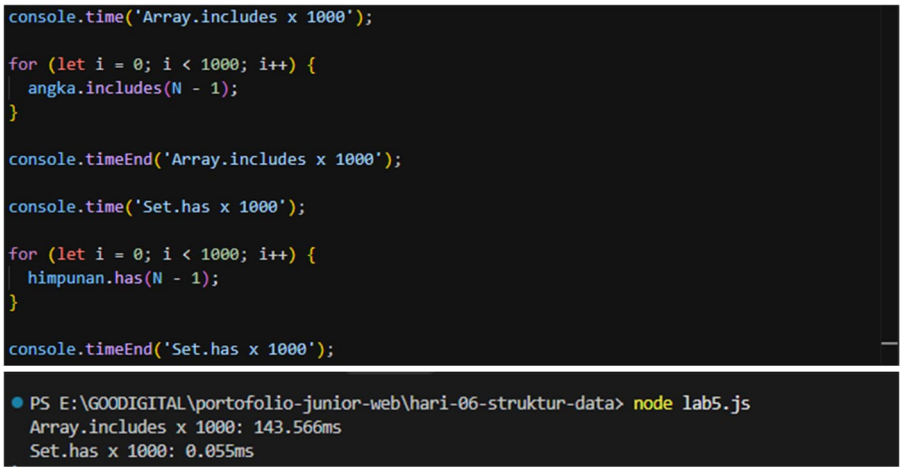
- Membuat file lab5.js.
- Menyiapkan data Array dan Set.
- Menyiapkan pengujian efisiensi struktur data.



```
hari-06-struktur-data > JS lab5.js > ...
1   import { Queue } from './structures.js';
2
3   const N = 100000;
4
5   const angka =
6     Array.from(
7       { length: N },
8       (_, i) => i
9     );
10
11  const himpunan =
12    new Set(angka);
```

2. Array.includes dan Set.has

- Mengukur pencarian data pada Array dan Set.
- Membandingkan hasil pengukuran.



```
console.time('Array.includes x 1000');

for (let i = 0; i < 1000; i++) {
  angka.includes(N - 1);
}

console.timeEnd('Array.includes x 1000');

console.time('Set.has x 1000');

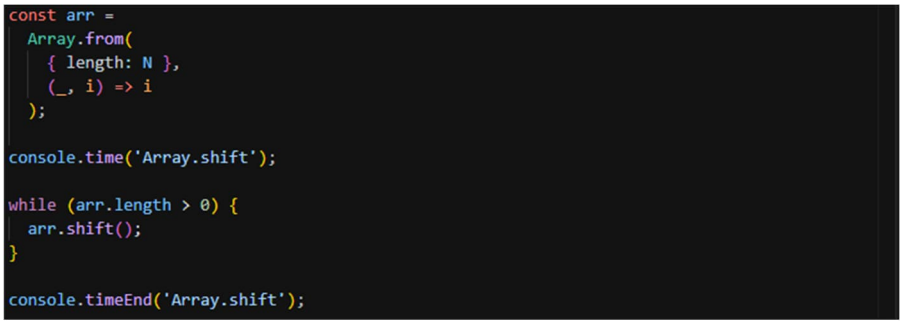
for (let i = 0; i < 1000; i++) {
  himpunan.has(N - 1);
}

console.timeEnd('Set.has x 1000');
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node lab5.js
Array.includes x 1000: 143.566ms
Set.has x 1000: 0.055ms
```

3. Array.shift dan Queue.dequeue

- Mengukur penghapusan data pada Array dan Queue.
- Membandingkan hasil pengukuran.



```
const arr =
  Array.from(
    { length: N },
    (_, i) => i
  );

console.time('Array.shift');

while (arr.length > 0) {
  arr.shift();
}

console.timeEnd('Array.shift');
```

```
const arr =
  Array.from(
    { length: N },
    (_, i) => i
  );

console.time('Array.shift');

while (arr.length > 0) {
  arr.shift();
}

console.timeEnd('Array.shift');

const q = new Queue();

for (let i = 0; i < N; i++) {
  q.enqueue(i);
}

console.time('Queue.dequeue');

while (!q.isEmpty()) {
  q.dequeue();
}

console.timeEnd('Queue.dequeue');
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node lab5.js
Array.includes x 1000: 105.894ms
Set.has x 1000: 0.033ms
Array.shift: 1.079s
Queue.dequeue: 13.014ms
```

```
PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node lab5.js
Array.includes x 1000: 105.661ms
Set.has x 1000: 0.034ms
Array.shift: 1.041s
Queue.dequeue: 6.981ms
PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node lab5.js
Array.includes x 1000: 155.041ms
Set.has x 1000: 0.033ms
Array.shift: 1.034s
Queue.dequeue: 7.186ms
PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node lab5.js
Array.includes x 1000: 105.025ms
Set.has x 1000: 0.032ms
Array.shift: 1.049s
Queue.dequeue: 7.404ms
```

Operasi	Waktu (ms)	Big-O
Array.includes x 1000	121.909	O(n) per pencarian
Set.has x 1000	0.033	O(1) per pencarian
Array.shift hingga kosong	1041.333	O(n) per operasi
Queue.dequeue hingga kosong	7.190	O(1) per operasi

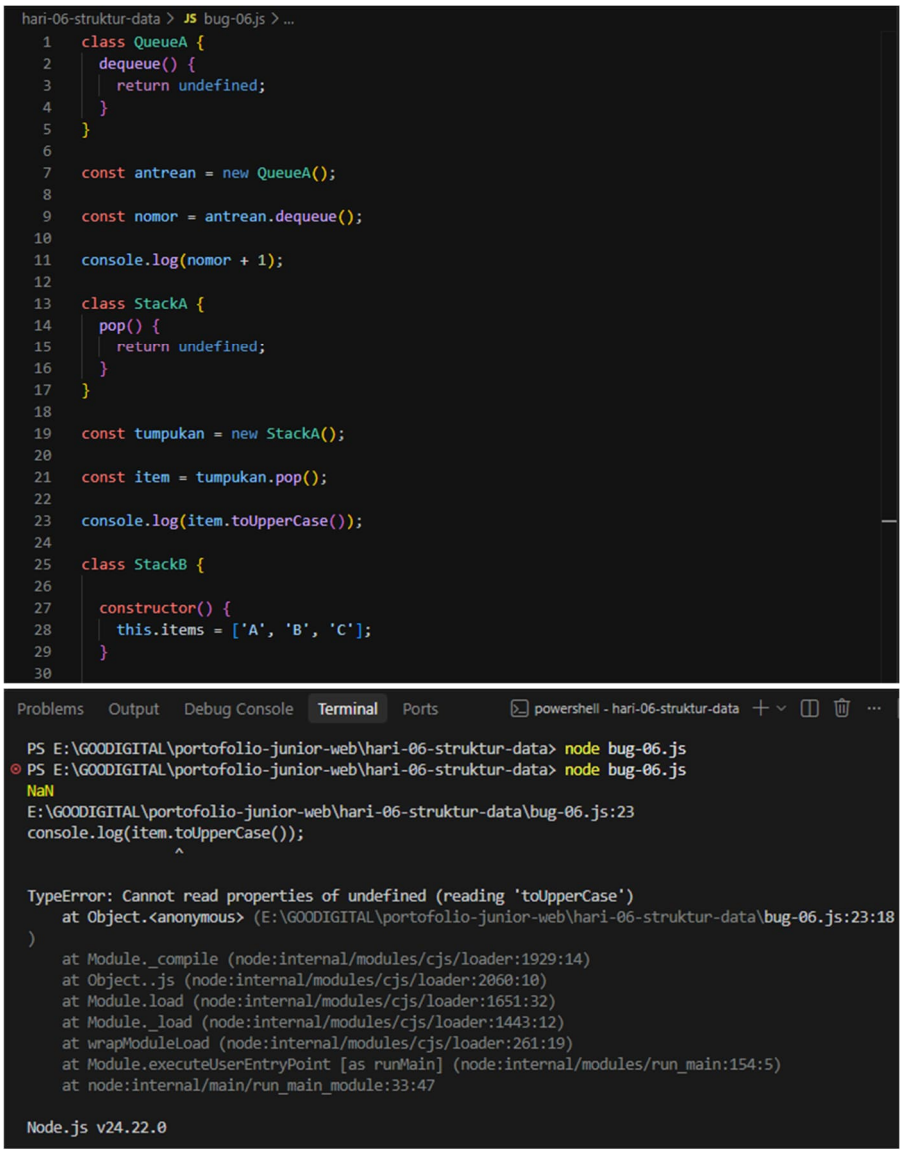
Kesimpulan:

Hasil pengujian menunjukkan bahwa Set.has() lebih cepat daripada Array.includes(), dan Queue.dequeue() jauh lebih cepat daripada Array.shift(). Hasil ini sesuai dengan teori Big-O yang menjelaskan bahwa operasi O(1) lebih efisien dibandingkan O(n) saat jumlah data bertambah besar.

Bug Hunt

1. Analisis Struktur Data

- Mengidentifikasi bug pada Stack, Queue, Array, dan pencarian data.
- Menganalisis penyebab bug.
- Memperbaiki kode yang bermasalah.
- Menguji hasil perbaikan.



```
hari-06-struktur-data > JS bug-06.js > ...
1   class QueueA {
2     dequeue() {
3       return undefined;
4     }
5   }
6
7   const antrean = new QueueA();
8
9   const nomor = antrean.dequeue();
10
11  console.log(nomor + 1);
12
13  class StackA {
14    pop() {
15      return undefined;
16    }
17  }
18
19  const tumpukan = new StackA();
20
21  const item = tumpukan.pop();
22
23  console.log(item.toUpperCase());
24
25  class StackB {
26
27    constructor() {
28      this.items = ['A', 'B', 'C'];
29    }
30
```

```
Problems  Output  Debug Console  Terminal  Ports   powershell - hari-06-struktur-data

PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node bug-06.js
PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node bug-06.js
NaN
E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data\bug-06.js:23
console.log(item.toUpperCase());
                 ^

TypeError: Cannot read properties of undefined (reading 'toUpperCase')
    at Object.<anonymous> (E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data\bug-06.js:23:18
)
    at Module._compile (node:internal/modules/cjs/loader:1929:14)
    at Object..js (node:internal/modules/cjs/loader:2060:10)
    at Module.load (node:internal/modules/cjs/loader:1651:32)
    at Module._load (node:internal/modules/cjs/loader:1443:12)
    at wrapModuleLoad (node:internal/modules/cjs/loader:261:19)
    at Module.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:154:5)
    at node:internal/main/run_main_module:33:47

Node.js v24.22.0
```

2. Pengujian Hasil Perbaikan

- Menjalankan program setelah perbaikan.
- Menguji Stack kosong.
- Menguji Queue kosong.
- Menguji hasil sorting dan pencarian data.



```
hari-06-struktur-data > JS bug-06.js > ...
// Bug 1
class QueueA {
  dequeue() {
    return undefined;
  }

  isEmpty() {
    return true;
  }
}

const antrean = new QueueA();

if (!antrean.isEmpty()) {
  const nomor = antrean.dequeue();
  console.log(nomor + 1);
} else {
  console.log('Antrean kosong');
}

// Bug 2
class StackA {
  pop() {
    return undefined;
  }
}

const tumpukan = new StackA();

const item = tumpukan.pop();
```

```
Problems  Output  Debug Console  Terminal  Ports   powershell - hari-06-struktur-data

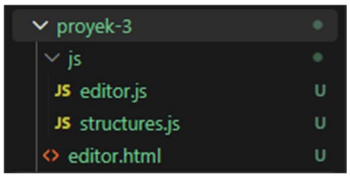
PS E:\GOODIGITAL\portofolio-junior-web\hari-06-struktur-data> node bug-06.js
Antrean kosong
Stack kosong
C
3
[ 'C', 'A', 'B' ]
Selesai
```

No	Kode bermasalah	Penyebab	Perbaikan
1	dequeue() pada queue kosong hasilnya dipakai sebagai angka	undefined atau NaN pada tampilan	Periksa isEmpty() sebelum memakai hasil
2	pop() stack kosong tanpa pemeriksaan	undefined merambat ke logika lain	Kembalikan nilai aman dan beri pesan
3	peek() memakai items[items.length]	Selalu undefined	items[items.length - 1]
4	size() pada Queue menghitung tail + head	Ukuran salah	tail - head
5	const salinan = daftar; salinan.sort()	Array asli ikut berubah	[...daftar].sort(...)
6	Mencari id dengan find di dalam loop 100.000 kali	Lambat, $O(n^2)$	Buat Map/objek berindeks id lebih dulu

PBL Hari 6 - Undo/Redo dan Antrean Loket (Bagian 1 Proyek 3)

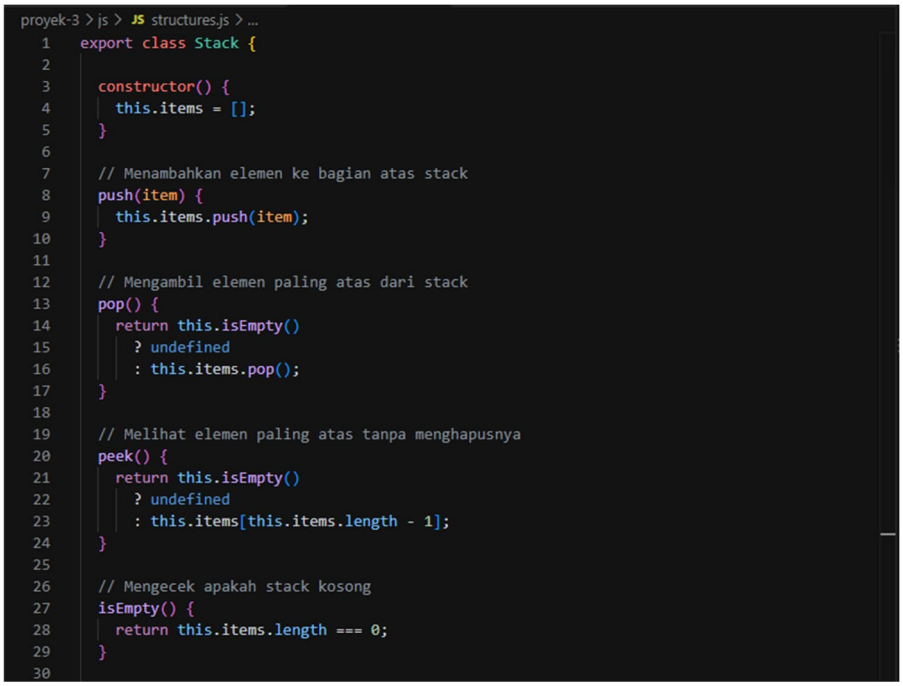
1. Menyiapkan Struktur Proyek

- Membuat folder proyek-3 dan js.
- Membuat file editor.html, editor.js, dan structures.js.



2. Membuat Class Stack dan Queue

- Menambahkan class Stack.
- Menambahkan class Queue.
- Mengekspor class untuk digunakan pada proyek.



3. Membuat Antarmuka Editor Undo/Redo

- Membuat file editor.html.
- Membuat area editor teks.
- Menambahkan tombol Undo dan Redo.
- Menambahkan informasi jumlah riwayat.

proyek-3 > editor.html > ...

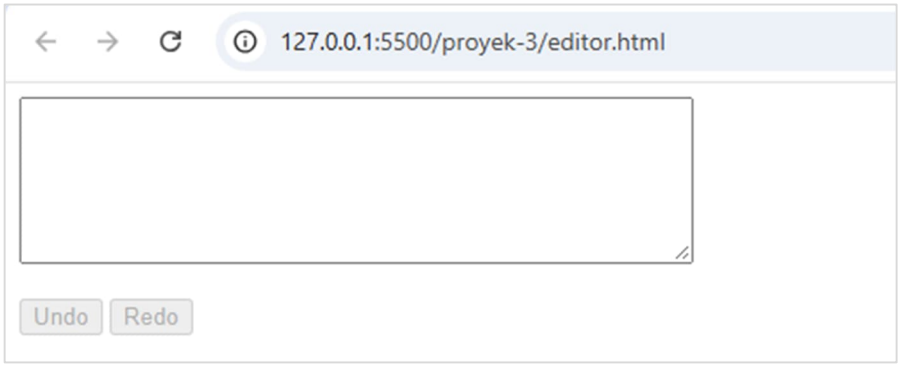
```html
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Editor Undo Redo</title>
</head>
<body>

  <textarea
    id="editor"
    rows="6"
    cols="50">
  </textarea>

  <p>

    <button
      id="undo"
      disabled>
      Undo
    </button>

    <button
      id="redo"
      disabled>
      Redo
    </button>
```



4. Membuat Logika Undo/Redo

- Mengimpor class Stack.
- Membuat Stack untuk Undo dan Redo.
- Menambahkan fungsi Undo dan Redo.
- Menampilkan informasi jumlah riwayat.

proyek-3 > js > editor.js > ...

```javascript
import { Stack } from './structures.js';

const editor = document.getElementById('editor');
const tombolUndo = document.getElementById('undo');
const tombolRedo = document.getElementById('redo');
const info = document.getElementById('info');

const undoStack = new Stack();
const redoStack = new Stack();

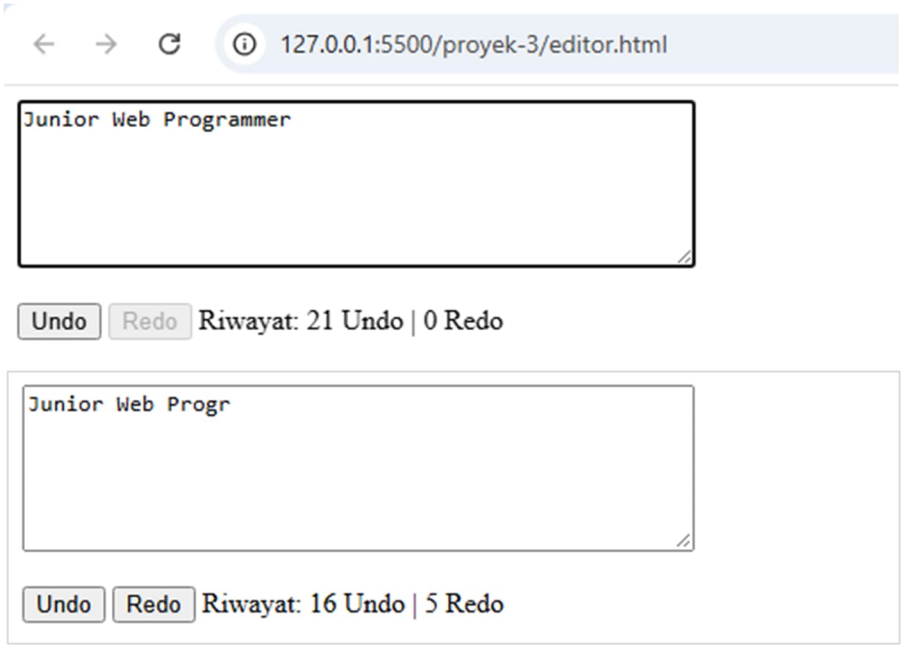
let teksSekarang = '';

function perbaruiTombol() {

  tombolUndo.disabled =
    undoStack.isEmpty();

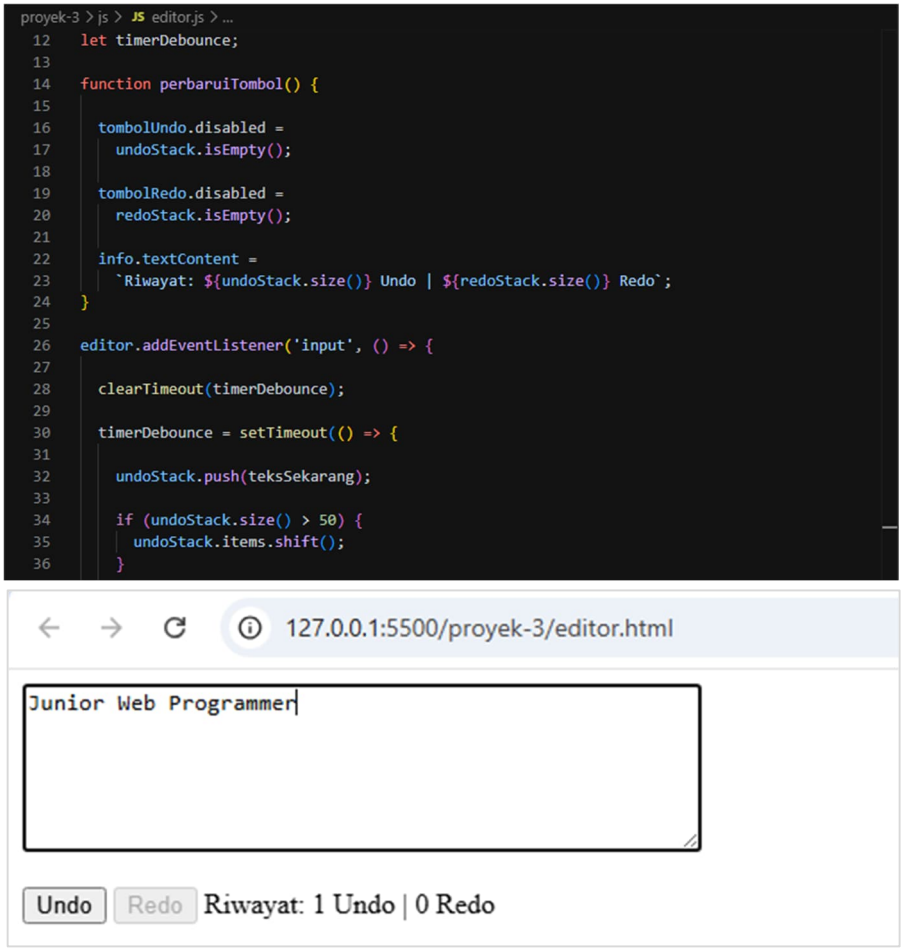
  tombolRedo.disabled =
    redoStack.isEmpty();

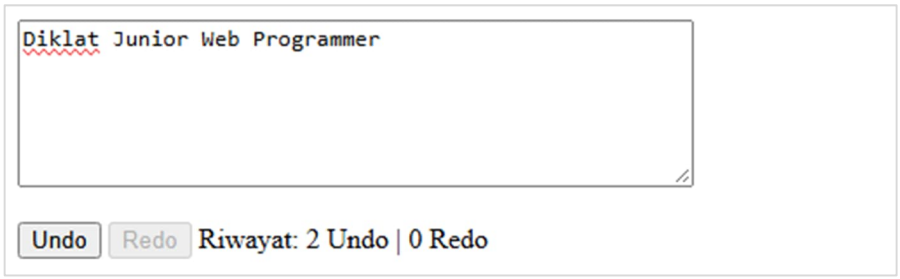
  info.textContent =
    `Riwayat: ${undoStack.size()} | Undo: ${redoStack.size()} | Redo:`;
}
```



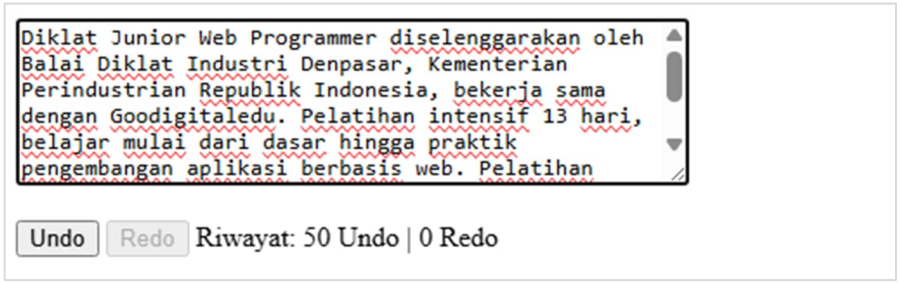
5. Tantangan Undo/Redo (Maksimal 50 Riwayat dan Debounce)

- Membatasi riwayat maksimal 50 langkah.
- Menambahkan debounce saat mengetik.
- Menguji penyimpanan riwayat.









6. Membuat Antarmuka Antrean Loket

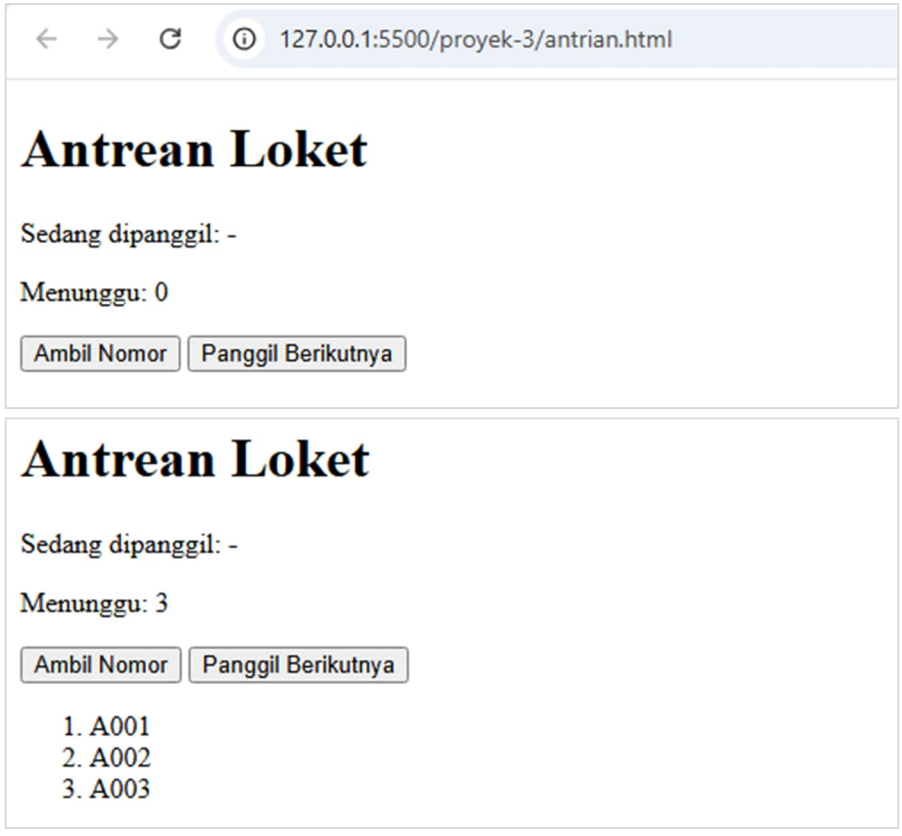
- Membuat file antrian.html.
- Menampilkan nomor yang sedang dipanggil.
- Menampilkan jumlah antrean.
- Menambahkan tombol Ambil Nomor dan Panggil Berikutnya.



7. Membuat Logika Antrean Loket

- Membuat file antrian.js.
- Menggunakan class Queue.
- Menambahkan fitur Ambil Nomor.
- Menambahkan fitur Panggil Berikutnya.
- Menampilkan daftar antrean dan jumlah antrean.

```
proyek-3 > js > JS antrian.js > ...
1  import { Queue } from './structures.js';
2
3  const antrian = new Queue();
4
5  let nomorTerakhir = 0;
6
7  const dipanggil =
8    document.getElementById('dipanggil');
9
10 const jumlah =
11   document.getElementById('jumlah');
12
13 const daftar =
14   document.getElementById('daftar');
15
16 function formatNomor(angka) {
17   return 'A' + String(angka).padStart(3, '0');
18 }
19
20 function tampilkanDaftar() {
21
22   daftar.textContent = '';
23
24   for (
25     let i = antrian.head;
26     i < antrian.tail;
27     i++
28   ) {
```





8. Menambahkan Riwayat Nomor yang Dipanggil

- Menambahkan riwayat nomor yang sudah dipanggil.
- Membatasi riwayat maksimal 5 data.
- Menampilkan riwayat pada halaman.

```
<button id="panggil">
  Panggil Berikutnya
</button>

<ol id="daftar"></ol>

<h2>Riwayat Panggilan</h2>

<ol id="riwayat"></ol>
```

```
function tampilkanRiwayat() {

 daftarRiwayat.textContent = '';

 for (const nomor of riwayat) {

   const li =
     document.createElement('li');

   li.textContent = nomor;

   daftarRiwayat.append(li);
 }
}
```





9. Menambahkan Tombol Reset Antrean

- Menambahkan tombol reset.
- Menghapus seluruh antrean aktif.
- Mengatur ulang nomor antrean.
- Mengosongkan riwayat panggilan.

proyek-3 > antrian.html > ... Text Editor

```
<button id="ambil">
  Ambil Nomor
</button>

<button id="reset">
Reset
</button>

<button id="panggil">
  Panggil Berikutnya
</button>
```

proyek-3 > js > antrian.js > ...

```
tombolReset.addEventListener('click', () => {

antrian.items = {};
antrian.head = 0;
antrian.tail = 0;

nomorTerakhir = 0;

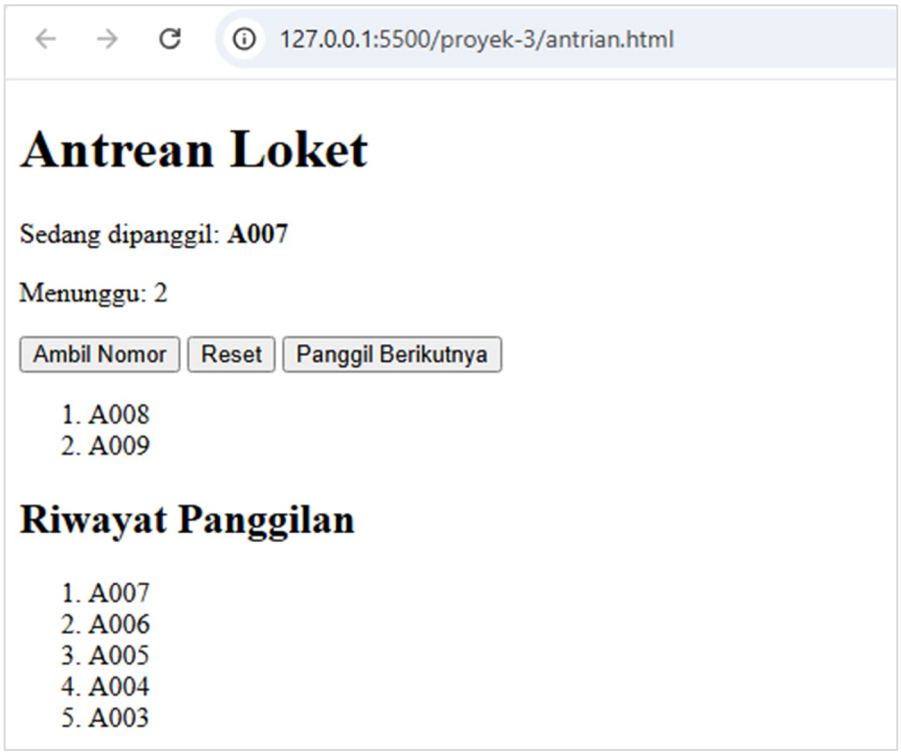
riwayat.length = 0;

dipanggil.textContent = '-';

tampilkanDaftar();
tampilkanRiwayat();

});

```





10. Analisis Pemilihan Struktur Data

Studi Kasus 1 - Daftar Kontak Ponsel dengan Pencarian Nama

Bagian	Isi
Masalah	Menyimpan dan mencari data kontak berdasarkan nama.
Data	Nama, nomor telepon, dan informasi kontak lainnya. Jumlah data dapat mencapai ratusan atau ribuan kontak.
Operasi dominan	Pencarian kontak berdasarkan nama dan penambahan kontak baru.
Alternatif	Array, Object/Map
Perbandingan	Array mudah digunakan tetapi pencarian harus menelusuri data satu per satu. Object/Map memungkinkan pencarian langsung berdasarkan key.
Keputusan	**Object/Map** dipilih karena pencarian berdasarkan nama lebih cepat dan efisien.

Studi Kasus 2 - Riwayat Halaman yang Dikunjungi (Back dan Forward)

Bagian	Isi
Masalah	Menyimpan riwayat halaman yang dikunjungi pengguna.
Data	URL atau alamat halaman yang dikunjungi. Jumlah data berubah sesuai aktivitas pengguna.
Operasi dominan	Menambahkan halaman baru, kembali ke halaman sebelumnya, dan maju ke halaman berikutnya.
Alternatif	Array, Stack
Perbandingan	Array dapat digunakan tetapi pengelolaan Back dan Forward lebih rumit. Stack mendukung mekanisme Last In First Out (LIFO) secara langsung.
Keputusan	**Stack** dipilih untuk fitur Back. Tombol Forward dapat menggunakan Stack kedua untuk menyimpan halaman yang telah di-Undo.

Studi Kasus 3 - Antrean Pasien Klinik dengan Kasus Darurat

Bagian	Isi
Masalah	Mengelola antrean pasien dengan kemungkinan adanya pasien darurat yang harus didahulukan.
Data	Data pasien dan tingkat prioritas pelayanan. Jumlah data berubah setiap saat.
Operasi dominan	Menambah pasien dan memanggil pasien untuk dilayani.
Alternatif	Queue, Priority Queue
Perbandingan	Queue melayani berdasarkan urutan kedatangan. Priority Queue dapat memberikan prioritas kepada pasien darurat.
Keputusan	**Priority Queue** dipilih karena pasien darurat harus dapat dilayani lebih dulu tanpa menunggu antrean normal.

11. Review

Butir	Ya/Tidak	Catatan
Struktur data dan UI dipisah berkas (modul)	☑ Ya	Logika dan tampilan sudah dipisah ke beberapa file sehingga lebih rapi dan mudah dicari.
Struktur kosong dan satu elemen sudah diuji	☑ Ya	Pengujian stack dan queue kosong sudah dilakukan, hasilnya sesuai harapan.
Tidak ada akses langsung ke properti internal dari luar kelas	☑ Ya	Sudah pakai toArray() pada Queue sehingga tidak perlu mengakses head, tail, dan items secara langsung.

Nama method konsisten (push, pop, enqueue, dequeue)	☑ Ya	Penamaan method sudah konsisten dan mudah dipahami.
Alasan pemilihan struktur data tertulis	☑ Ya	Sudah ada pada dokumen analisis struktur data di bagian PBL.

12. Repository GitHub

- Melakukan commit.
- Melakukan push ke GitHub.
- Tautan repository: https://github.com/amribaros15/portofolio-junior-web/

